



CONSISTENCIA E INTEGRIDAD EN BASES DE DATOS

Objetivo

Este documento abarca la Consistencia e Integridad de Datos en Bases de Datos, definiendo la Integridad de Datos como un control de calidad esencial frente a amenazas físicas y lógicas. Explica cómo aplicar la integridad mediante restricciones como NOT NULL y PRIMARY KEY, y aborda los Triggers (Disparadores). Finalmente, define la Consistencia en Bases de Datos como la garantía de que los datos son precisos y uniformes, manteniendo la base de datos en un estado válido bajo el modelo ACID.

Índice

Integridad y Consistencia de Datos.....	2
Integridad en Bases de Datos.....	3
Aplicación de integridad - Restricciones en tablas.....	5
Aplicación de integridad - Trigger.....	5
Otras aplicaciones de integridad.....	8
Consistencia en Bases de Datos.....	9
Bibliografía.....	10



Integridad y Consistencia de Datos

Introducción

La integridad de los datos es similar al control de calidad en las empresas tradicionales orientadas al producto y garantiza que la materia prima es correcta, segura y adecuada para el uso previsto.

Es de suma importancia que los datos sean coherentes para que las empresas puedan obtener información fiable y fehaciente desde la cual tomar decisiones, observar registros históricos y conocer el estado de situación de la organización.

La frase "basura entra, basura sale" nos revela que los datos erróneos, una vez operacionalizados, conducen a resultados indeseables.

Factores que afectan la integridad y consistencia en una base de datos

La integridad y la consistencia de los datos son fundamentales para cualquier organización, pero enfrentan una variedad de amenazas que pueden poner en peligro su fiabilidad. Estas amenazas se pueden clasificar en dos grandes categorías: físicas y lógicas.

Las amenazas físicas abarcan todos aquellos eventos que pueden causar daño directo al hardware o a los medios de almacenamiento donde residen los datos. Los desastres naturales, como inundaciones, terremotos o tormentas, pueden destruir equipos y causar pérdida irreversible de datos, como también las condiciones ambientales extremas, como la humedad, el calor o el polvo. También debemos tener en cuenta posibles accidentes que pueden ocurrir. Tanto incendios como cortes de energía o sobrecargas eléctricas pueden dañar el hardware y provocar la pérdida de datos.

Por otro lado, las amenazas lógicas son aquellas que surgen de errores en el software, procesos o acciones humanas que pueden comprometer la integridad y consistencia de los datos. Los errores humanos generados al momento de ingresar datos, como errores tipográficos, omisiones o entradas incorrectas, pueden introducir datos erróneos en el sistema. Las fallas durante la ejecución de procesos, como actualizaciones incompletas o transacciones interrumpidas, pueden dejar los datos en un estado inconsistente. El acceso no autorizado a los datos, debido a la falta de seguridad, puede permitir que usuarios malintencionados modifiquen, eliminen o roben datos, lo que compromete su integridad y confidencialidad. Por último, los virus, malware y otros programas maliciosos también pueden dañar o corromper los datos, o incluso cifrarlos para exigir un rescate.



Por estos factores afirmamos que la integridad y consistencia de los datos son esenciales para proteger los datos de cualquier organización y garantizar su fiabilidad a largo plazo, logrando mejorar la eficacia en la toma de decisiones informadas y el funcionamiento de la organización.

Integridad en Bases de Datos

Definición

La integridad de datos hace referencia al conjunto de procesos, reglas y normas que tienen como fin mantener los datos completos, precisos, coherentes y seguros a lo largo de su ciclo de vida.

La intención es mantener todos los elementos de los datos intactos, sin alteraciones, truncamientos ni pérdidas, evitando cambios que podrían distorsionar el análisis y poner en peligro la coherencia de la información. Sin procesos de integridad de los datos, las organizaciones no podrían comprobar ni confiar en su veracidad. Al crear una base de datos, se debe prestar atención a la integridad de los datos y a cómo mantenerlos, asegurando que permanezcan intactos y sin cambios de vida. Esto incluye la captura de los datos, el almacenamiento, las actualizaciones, su estabilidad, el rendimiento, las transferencias, la capacidad de recuperación, etc.

Tipos de Integridad de datos

- **Integridad Física:** Es la protección de los datos en su formato físico. Ésta puede ser comprometida cuando tiene lugar una catástrofe natural, un evento como un apagón o un daño físico accidental o intencional.
- **Integridad Lógica:** También se enfoca en la protección de los datos, pero desde el punto de vista lógico. Se puede distinguir en 4 tipos de integridades distintas:
 - **Integridad de dominio:** Un dominio está definido por un conjunto específico de valores para las columnas de una tabla, incluidas las restricciones y reglas que rigen la cantidad, el formato y los datos que pueden introducirse. La integridad del dominio ayuda a garantizar la precisión de los elementos de datos dentro de un dominio.

```
CREATE TABLE clientes (  
    id INT NOT NULL,  
    nombre VARCHAR(255) NOT NULL,  
    edad INT NOT NULL,  
    email VARCHAR(255) UNIQUE,
```



```
ciudad_id INT NOT NULL,  
);
```

- **Integridad de la identidad:** Se basa en claves y valores únicos creados para identificar los datos, lo que garantiza que los mismos datos no aparezcan en la lista varias veces y que los campos de la tabla se rellenen correctamente.

```
CREATE TABLE clientes (  
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(255),  
    edad INT,  
    email VARCHAR(255),  
    ciudad_id INT  
);
```

- **Integridad referencial:** Una serie de procesos que garantizan que los datos se almacenen y utilicen de manera uniforme. Las estructuras de bases de datos incorporan reglas que imponen la presencia de registros coincidentes en las tablas vinculadas, lo que evita los registros huérfanos y mantiene la coherencia de los datos en toda la base de datos.

```
CREATE TABLE clientes (  
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(255) NOT NULL,  
    edad INT NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    ciudad_id INT NOT NULL,  
    FOREIGN KEY (ciudad_id) REFERENCES ciudades(id)  
);
```

- **Integridad definida por el usuario:** Cuando los usuarios crean reglas y restricciones en torno a los datos para alinearlos con sus especificaciones únicas. Este método se emplea generalmente con otros procesos que no garantizan la seguridad de los datos.

```
CREATE TABLE clientes (  
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(255) NOT NULL,  
    edad INT NOT NULL CHECK (edad >= 18),  
    email VARCHAR(255) UNIQUE,
```



```
ciudad_id INT NOT NULL,  
FOREIGN KEY (ciudad_id) REFERENCES ciudades(id)  
CONSTRAINT cliente_mayor_de_edad CHECK (edad >= 18)  
);
```

Aplicación de integridad - Restricciones en tablas

Las restricciones que se utilizan para limitar los datos que puede recibir una columna de una tabla se realizan mediante la cláusula **CONSTRAINT**. Las restricciones más comunes son:

- **NOT NULL**: Garantizando que un campo específico no posea valores NULL.

```
CREATE TABLE Productos (  
    [...]  
    nombre VARCHAR(100) NOT NULL  
);
```

- **UNIQUE**: Garantiza que cada valor es único en la columna; es decir, no puede haber dos registros con dos valores iguales en la columna.

```
CREATE TABLE Empleados (  
    [...]  
    email VARCHAR(100) UNIQUE  
);
```

- **PRIMARY KEY**: Garantiza que cada registro en una tabla tenga un identificador único. Las claves primarias deben contener valores **UNIQUE** y no pueden contener valores **NULL**. Una tabla puede tener solo **UNA** clave primaria; y en la tabla, esta clave primaria puede consistir en columnas únicas o múltiples (campos).

```
CREATE TABLE Usuarios (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    [...]  
);  
  
CREATE TABLE Contrato (  
    id_contrato INT,  
    id_vendedor INT,
```



```
[...]  
PRIMARY KEY (id_contrato, id_vendedor)  
);
```

- **FOREIGN KEY:** Garantiza la correcta asociación de una tabla con otra, evitando acciones que destruyan los enlaces entre tablas.

```
CREATE TABLE Pedidos (  
    [...]  
    usuario_id INT,  
    [...]  
    FOREIGN KEY (usuario_id) REFERENCES Usuarios(id)  
);
```

- **CHECK:** Permite establecer condiciones específicas sobre los valores de un campo.

```
CREATE TABLE Estudiantes (  
    [...]  
    edad INT CHECK (edad >= 18)  
);  
  
CREATE TABLE Estudiantes (  
    [...]  
    edad INT,  
    CHECK (edad >= 18)  
);
```

- **DEFAULT:** Si no se especifica el valor del campo al momento de agregar un nuevo registro, entonces se asignará el valor indicado en la cláusula DEFAULT. También, se puede hacer uso de esta palabra reservada para hacer referencia al valor asociado.

```
CREATE TABLE Ordenes (  
    [...]  
    fecha DATE DEFAULT CURRENT_DATE  
);
```



Aplicación de integridad - Trigger

Definición

Un Trigger (disparador) es un objeto de la Base de Datos que está asociado con una tabla, y que se activa cuando se produce un evento particular en ella. En definitiva, Un Trigger es una acción en cadena que empieza cuando un evento específico ocurre sobre una tabla específica.

Los eventos disparadores pueden ser tres: INSERT, DELETE y UPDATE.

Un Trigger puede activarse automáticamente antes (BEFORE) o después (AFTER) de que ocurra el evento.

Tipos de Triggers

- Disparadores de fila (Row Triggers): Se ejecutan una vez para cada fila afectada por una instrucción DML. Se crean utilizando la cláusula for each row en el comando create trigger
- Disparadores de Sentencia (Statement Triggers): Se ejecuta solo una vez, independientemente de la cantidad de filas que afecte la instrucción DML.
Importante: Este tipo de trigger no está disponible para MySQL, si para otros gestores.

Limitaciones en la sentencia de un Trigger

El disparador no puede referirse a tablas directamente por su nombre, incluyendo la misma tabla a la que está asociado. Sin embargo, se pueden emplear las palabras clave OLD y NEW.

- OLD se refiere a un registro existente que va a borrarse o que va a actualizarse antes de que esto ocurra.
- NEW se refiere a un registro nuevo que se insertará o a un registro modificado luego de que ocurra la modificación.

El disparador no puede invocar procedimientos almacenados utilizando la sentencia CALL. (Esto significa, por ejemplo, que no se puede utilizar un procedimiento almacenado para eludir la prohibición de referirse a tablas por su nombre).

El disparador no puede utilizar sentencias que inicien o finalicen una transacción, tal como START TRANSACTION, COMMIT, o ROLLBACK



Gestión de Errores Durante la Ejecución de un Trigger (MYSQL)

- Si lo que falla es un disparador BEFORE, no se ejecuta la operación en el registro correspondiente.
- Un disparador AFTER se ejecuta solamente si el disparador BEFORE (de existir) y la operación se ejecutaron exitosamente.
- Un error durante la ejecución de un disparador BEFORE o AFTER deriva en la falla de toda la sentencia que provocó la invocación del disparador.
- En tablas transaccionales, la falla de un disparador (y por lo tanto de toda la sentencia) debería causar la cancelación (ROLLBACK) de todos los cambios realizados por esa sentencia. En tablas no transaccionales, cualquier cambio realizado antes del error no se ve afectado

Estructura Básica de un Trigger

Llamada de activación: es la sentencia que permite “disparar” el código a ejecutar.

Restricción: es la condición necesaria para ejecutar el código. Esta restricción puede ser de tipo condicional o de tipo nulidad.

Acción a ejecutar: es la sentencia de instrucciones a ejecutar una vez que se han cumplido las condiciones iniciales.

```
CREATE TRIGGER [NOMBRE_TRIGGER] [AFTER|BEFORE]
[INSERT|UPDATE|DELETE] ON [NOMBRE_TABLA] FOR EACH ROW
[SENTENCIA]
```

La tabla a la que está asociada debe ser una tabla permanente, no puede ser una tabla temporal ni una vista.

Si se desean ejecutar múltiples sentencias, deben colocarse entre BEGIN... END, el constructor de sentencias compuestas.

Esto además posibilita emplear las mismas sentencias permitidas en rutinas almacenadas.

¿Cuándo usar Trigger?

Los Triggers los podemos implementar en varios casos de uso para que mantengan la integridad de la base de datos, como por ejemplo Validar información, Calcular atributos derivados, Seguimientos de movimientos en la base de datos, etc.



Cuando surja una necesidad en donde veas que necesitas que se ejecute una acción implícitamente (sin que la ejecutes manualmente) sobre los registros de una tabla, entonces puedes considerar el uso de un Trigger.

¿Cómo se usan los Trigger?

Para crear un Trigger se utiliza la sentencia CREATE TRIGGER, junto a las cláusulas que le indican cuándo activarse y qué hacer cuando se active.

Si por alguna razón, quisiéramos que el Trigger no se ejecute más, debemos eliminarlo. Para eliminar un Trigger, se utiliza la sentencia DROP TRIGGER indicando el nombre del Trigger a borrar.

Comparación entre el uso de un Trigger y un Check

Como se puede observar, existen casos donde un check puede ser implementado como un trigger y viceversa. Aun así, existen algunas reglas generales que permiten elegir la mejor forma de implementar una restricción de integridad.

Para validaciones sencillas que afectan una sola fila, la opción preferente es utilizar el CHECK. Este método es ideal cuando la condición de validación es directa y se aplica sobre una columna, afectando a todos los registros de la tabla.

Para validaciones más complejas, que involucran datos de otras tablas, o necesitan considerar múltiples filas para determinar la validez, se debe recurrir a los TRIGGER. Estos ofrecen mayor flexibilidad al permitir ejecutar lógica más elaborada antes o después de una operación de base de datos.

Es importante destacar que, siempre que sea posible, se debe optar por CHECK en lugar de TRIGGER, ya que los CHECK son generalmente más eficientes y tienen un menor impacto en el rendimiento de la base de datos. Los TRIGGER solo deben emplearse como último recurso, cuando las capacidades de CHECK resultan insuficientes para la validación requerida.

Característica	CHECK	TRIGGER
Nivel de aplicación	Columna	Fila
Tipo de Validación	Simple	Simple y complejas
Acciones adicionales	No	Sí, puede modificar tablas o lanzar errores personalizados
Impacto en el rendimiento	Bajo	Medio-Alto (dependiendo de la lógica)



Ventajas de los Triggers

- Seguridad de los datos mejorada:
 - Ofrecen chequeos de seguridad basados en valores.
- Integridad de los datos mejorada:
 - Fuerzan restricciones dinámicas de integridad de datos y de integridad referencial.
 - Aseguran que las operaciones relacionadas se realizan juntas de forma implícita.
 - Respuesta instantánea ante un evento auditado
 - Ofrece un mayor control sobre la B.D.

Desventajas de los Triggers

- Hay que definir con anticipación la tarea que realizará el Trigger
- Peligro de pérdida en Reorganizaciones
- Hay que programarlos para cada DBMS
- Un Trigger nunca se llama directamente.
- Los Triggers no se desarrollan pensando en un solo registro, los mismos deben funcionar en conjunto con los datos ya que se disparan por operación y no por registro.
- Por funcionalidad, no hay que poner en uno solo las funciones de INSERT, UPDATE y DELETE.
- Utilizar moderadamente los Triggers.
- No se pueden utilizar en tablas temporales

Otras aplicaciones de integridad

- Optimización de consultas
- Seguridad y control de acceso
- Mantenimiento y monitoreo de datos
- Plan estratégico de recuperación de base de datos



Consistencia en Bases de Datos

Definición de Consistencia de Datos

La consistencia de datos en bases de datos garantiza que los datos sean precisos, confiables y uniformes en todo momento y en todos los sistemas. Esto significa que todas las transacciones respetan las reglas y restricciones definidas, llevando la base de datos de un estado válido a otro, asegurando que los datos permanezcan inalterados y que todos los usuarios vean la misma información.

Importancia de la Consistencia de Datos

La consistencia de datos es vital para una amplia gama de aplicaciones y sistemas. Su importancia es especialmente pronunciada en entornos donde los datos son accedidos y modificados frecuentemente por múltiples usuarios o procesos. Los mecanismos de consistencia ayudan a prevenir conflictos que pueden surgir del acceso concurrente a los datos, garantizando así que los datos permanezcan precisos y fiables para la toma de decisiones, análisis y reportes.

Cómo Funciona la Consistencia de Datos

La consistencia en el modelo ACID garantiza que los datos en una base de datos respeten todas las reglas definidas. Cada transacción, que es una unidad de trabajo indivisible en una base de datos, debe dejar la base de datos en un estado válido. Si una transacción intenta violar alguna regla, se revierte por completo, asegurando así que la base de datos siempre refleje un estado coherente y confiable.



Bibliografía

Butti, Guereta, Hernández, Pereira (2016) Triggers - Stored Procedures - Functions. Apunte para Gestión de Datos de Ingeniería en Sistemas de Información de UTN FRRo.

<https://repositorio.usam.ac.cr/xmlui/bitstream/handle/11506/2124/LEC%20ING%20SIST%200068%202020.pdf?sequence=1&isAllowed=y>

<https://www.ibm.com/es-es/topics/data-integrity>

<https://www.astera.com/es/type/blog/data-integrity-in-a-database/>

<https://www.janbasktraining.com/tutorials/database-integrity/>

<https://www.vpnunlimited.com/es/help/cybersecurity/data-consistency>

<https://thedataschools.com/que-es/consistencia.html>